reference/python/uni_core_sdk.py (v8.2 — English Master Version)

```
#!/usr/bin/env python3
"""
.uni Format Reference SDK (Version 8.2 — Pure Binary Zero-Copy &
Holo-Stream Ready)
License: USPL-1.0 / CC-BY-4.0

Implements:
  - 32-byte O(1) Little-Endian UniHeader validation with Payload DoS
protection
  - Full 39-byte (UniHeader + NodeDesc) AEAD AES-256-GCM AAD
cryptographic binding
  - 100% binary Zero-JSON TLV serialization across 7 node types
(0x01..0x07) via memoryview
  - Fixed 141-byte binary NOTARY_ANCHOR (0x01) node
  - 32-byte AVX2/NEON cache-aligned KNOWLEDGE_GRAPH (0x03) binary
triplets
  - 20-byte binary HOLO_WAVEFRONT (0x07) voxels for Pockels crystals
and MEMS mirrors
  - RFC 5869 HKDF-SHA256 per-node key derivation and 256-bit SHA-256
Merkle Tree
  - Ed25519-anchored Selective Single-Node Disclosure (Merkle Branch
Proof verification)
  - Digital Notary Anchor (Delta-T cooling-off timelock and
Steganographic Duress Key routing)
  - Zone 2 Semantic Triage (Cosine Distance clustering and in-RAM
binary graph merging)
"""

import struct
import hashlib
import hmac
import math
import time
import os
from enum import IntEnum, IntFlag
from typing import Dict, List, Tuple, Any, Optional, Union, Iterator

# Attempt to load hardware-accelerated primitives from standard
'cryptography' package
try:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
    from cryptography.hazmat.primitives.asymmetric.ed25519 import (
        Ed25519PrivateKey,
        Ed25519PublicKey,
    )
```

```
    HAS_CRYPTO_LIB = True
except ImportError:
    HAS_CRYPTO_LIB = False


#
# ============================================================================
# 1. UNIVERSAL NODE TYPES, PROTOCOL FLAGS & MEMORY ZONES (.UNI V8.2)
#
# ============================================================================

class ProtoFlags(IntFlag):
    VERSION_8_2               = 0x0820
    TAMPER_MESH_INTACT        = 0x0001  # Hardware perimeter
tamper-mesh intact
    RS_FEC_APPENDED           = 0x0002  # Reed-Solomon RS(255,223)
parity tail appended
    STREAM_DELTA_FRAME        = 0x0004  # High-speed 120Hz Delta-Frame
(skips Node 0 Ed25519)
    SPATIAL_FRUSTUM_BROADCAST = 0x0008  # Facade-wide broadcast (crop
by Zone 0 coordinates)


class UniNodeType(IntEnum):
    NOTARY_ANCHOR     = 0x01  # System Node 0: Fixed 141B binary
Notary (Merkle + Ed25519 + Delta-T)
    SEMANTIC_TENSOR   = 0x02  # Bit-exact IEEE 754 float32 embedding
array for Zero-Parsing inference
    KNOWLEDGE_GRAPH   = 0x03  # 32-byte binary triplets (Subject,
Predicate, Object) with Delta flags
    TELEMETRY_MATRIX  = 0x04  # 8-byte binary sensor pairs (MetricID32
+ float32) for MZM-Pod & Facade
    BEHAVIOR_CONTRACT = 0x05  # Fixed 48-byte binary contract (TTL,
zone, HVAC mode, laser pulse cap)
    GENESIS_TEXT      = 0x06  # Human-readable Markdown/UTF-8 fallback
& legacy export layer
    HOLO_WAVEFRONT    = 0x07  # 20-byte binary voxel vectors (X, Y, Z,
Pockels 2*pi phase sawtooth)


class MemoryZone(IntEnum):
    ZONE_0_SOVEREIGN_VAULT = 0  # Strictly local root key enclave, V0
log & spatial frustum coords
    ZONE_1_ACTIVE_STREAM   = 1  # Volatile working RAM/SRAM buffer
with Dead Man's Switch
    ZONE_2_SEMANTIC_GRAPH  = 2  # Long-term vector & graph archive
(Cosine merging threshold 0.15)
    ZONE_3_QUARANTINE      = 3  # Isolated sandbox for untrusted
```

```
inputs & Duress Key alerts


class HVACMode(IntEnum):
    PASSIVE_IDLE          = 0  # Passive baseline state
    SOLAR_SHIELD_SUMMER   = 1  # Summer mode: 92% IR/UV reflection +
passive Venturi outdoor venting
    THERMAL_TRAP_WINTER   = 2  # Winter mode: waveguide photon-heat
retention inside glazing unit
    VENTURI_HVAC_RECOVERY = 3  # Kinematic coupling of heated Venturi
exhaust into building HVAC intake


class UIIntentCode(IntEnum):
    INTENT_BACKGROUND_PASSIVE = 0x0000
    INTENT_WEIGHT_FOCUS       = 0x0001
    INTENT_ALERT_CRITICAL     = 0x0002
    INTENT_HOLO_PARALLAX_3D   = 0x0003


#
======================================================================
# 2. DETERMINISTIC SEMANTIC & TELEMETRY HASHING (ZERO TEXT PARSING)
#
======================================================================

_SEMANTIC_ID64_REGISTRY: Dict[int, str] = {}
_METRIC_ID32_REGISTRY: Dict[int, str] = {}


def semantic_id64(concept: Union[int, str]) -> int:
    """Maps a concept string to a deterministic 64-bit unsigned
integer (Blake2b-64)."""
    if isinstance(concept, int):
        return concept & 0xFFFFFFFFFFFFFFFF
    digest = hashlib.blake2b(concept.encode("utf-8"),
digest_size=8).digest()
    val = struct.unpack("\x3cQ", digest)[0]
    _SEMANTIC_ID64_REGISTRY[val] = concept
    return val


def resolve_semantic_id64(val: int) -> str:
    """Returns the human concept label if registered locally,
otherwise hex ID64."""
    return _SEMANTIC_ID64_REGISTRY.get(val, f"0x{val:016x}")
```

```
def metric_id32(label: Union[int, str]) -> int:
    """Maps a sensor metric name to a deterministic 32-bit unsigned
integer (Blake2s-32)."""
    if isinstance(label, int):
        return label & 0xFFFFFFFF
    digest = hashlib.blake2s(label.encode("utf-8"),
digest_size=4).digest()
    val = struct.unpack("\x3cI", digest)[0]
    _METRIC_ID32_REGISTRY[val] = label
    return val


def resolve_metric_id32(val: int) -> str:
    """Returns the human sensor metric name if registered locally,
otherwise hex ID32."""
    return _METRIC_ID32_REGISTRY.get(val, f"0x{val:08x}")


#
=============================================================================
# 3. CRYPTOGRAPHIC CORE (RFC 5869 HKDF, AEAD AES-256-GCM, ED25519)
#
=============================================================================

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int =
32) -> bytes:
    """Derives isolated cryptographic subkeys pursuant to RFC 5869
HKDF-SHA256."""
    prk = hmac.new(salt, ikm, hashlib.sha256).digest()
    okm, prev = b"", b""
    steps = (length // 32) + 2
    for counter in range(1, steps):
        prev = hmac.new(prk, prev + info + struct.pack("B", counter),
hashlib.sha256).digest()
        okm += prev
    return okm[:length]


def aead_encrypt(
    key: bytes, iv: bytes, plaintext: Union[bytes, memoryview], aad:
bytes
) -> Tuple[bytes, bytes]:
    """Encrypts a node payload using AES-256-GCM (or stdlib
CTR+HMAC-SHA256 fallback)."""
    pt_bytes = bytes(plaintext)
    if HAS_CRYPTO_LIB:
        ct_tag = AESGCM(key).encrypt(iv, pt_bytes, aad)
        return ct_tag[:-16], ct_tag[-16:]
```

```
    enc_k = hmac.new(key, b"ENC" + iv, hashlib.sha256).digest()
    mac_k = hmac.new(key, b"MAC" + iv, hashlib.sha256).digest()
    ct = bytearray(len(pt_bytes))
    for i in range(0, len(pt_bytes), 32):
        ks = hashlib.sha256(enc_k + struct.pack("\x3cI", i //
32)).digest()
        chunk = pt_bytes[i:i + 32]
        for j, b in enumerate(chunk):
            ct[i + j] = b ^ ks[j]
    tag = hmac.new(mac_k, aad + iv + bytes(ct),
hashlib.sha256).digest()[:16]
    return bytes(ct), tag


def aead_decrypt(
    key: bytes,
    iv: Union[bytes, memoryview],
    ciphertext: Union[bytes, memoryview],
    tag: Union[bytes, memoryview],
    aad: bytes,
) -> bytes:
    """Verifies the 128-bit authentication tag (bound to 39B AAD) and
decrypts the node payload."""
    iv_b, ct_b, tag_b = bytes(iv), bytes(ciphertext), bytes(tag)
    if HAS_CRYPTO_LIB:
        return AESGCM(key).decrypt(iv_b, ct_b + tag_b, aad)
    enc_k = hmac.new(key, b"ENC" + iv_b, hashlib.sha256).digest()
    mac_k = hmac.new(key, b"MAC" + iv_b, hashlib.sha256).digest()
    exp_tag = hmac.new(mac_k, aad + iv_b + ct_b,
hashlib.sha256).digest()[:16]
    if not hmac.compare_digest(tag_b, exp_tag):
        raise ValueError("CRITICAL SECURITY ALERT: AEAD Tag mismatch!
Header or node tampered.")
    pt = bytearray(len(ct_b))
    for i in range(0, len(ct_b), 32):
        ks = hashlib.sha256(enc_k + struct.pack("\x3cI", i //
32)).digest()
        chunk = ct_b[i:i + 32]
        for j, b in enumerate(chunk):
            pt[i + j] = b ^ ks[j]
    return bytes(pt)


class SovereignKeyring:
    """
    Zone 0 Key Management Enclave.
    Maintains both a Primary Signing Key and a Steganographic Duress
Key.
```

```
    Both keys produce mathematically valid external signatures to
protect the human host,
    while trusted nodes silently route Duress-signed containers into
Zone 3 Quarantine.
    """
    def __init__(self, master_seed: bytes):
        self.master_encryption_secret = hkdf_sha256(master_seed,
b"Z0_SALT", b"ENC_MASTER")
        self._primary_seed = hkdf_sha256(master_seed, b"Z0_SALT",
b"ED25519_PRIMARY")
        self._duress_seed  = hkdf_sha256(master_seed, b"Z0_SALT",
b"ED25519_DURESS")

        if HAS_CRYPTO_LIB:
            self._primary_sk =
Ed25519PrivateKey.from_private_bytes(self._primary_seed)
            self._duress_sk  =
Ed25519PrivateKey.from_private_bytes(self._duress_seed)
            self.primary_pubkey =
self._primary_sk.public_key().public_bytes_raw()
            self.duress_pubkey  =
self._duress_sk.public_key().public_bytes_raw()
        else:
            self.primary_pubkey = hashlib.sha256(b"PUB:" +
self._primary_seed).digest()
            self.duress_pubkey  = hashlib.sha256(b"PUB:" +
self._duress_seed).digest()

    def sign_data(self, message: bytes, under_duress: bool = False) ->
Tuple[bytes, bytes]:
        """Returns (signer_pubkey_32B, signature_64B)."""
        if HAS_CRYPTO_LIB:
            sk = self._duress_sk if under_duress else self._primary_sk
            pub = self.duress_pubkey if under_duress else
self.primary_pubkey
            return pub, sk.sign(message)
        priv = self._duress_seed if under_duress else
self._primary_seed
        pub = self.duress_pubkey if under_duress else
self.primary_pubkey
        sig_p1 = hmac.new(priv, message, hashlib.sha256).digest()
        sig_p2 = hashlib.sha256(pub + message + sig_p1).digest()
        return pub, sig_p1 + sig_p2

    @staticmethod
    def verify_signature(pubkey: bytes, message: bytes, signature:
bytes) -> bool:
        """Public mathematical signature verification (returns True
```

```
for both Primary & Duress)."""
        if len(signature) != 64 or len(pubkey) != 32:
            return False
        if HAS_CRYPTO_LIB:
            try:

Ed25519PublicKey.from_public_bytes(pubkey).verify(signature, message)
                return True
            except Exception:
                return False
        exp_p2 = hashlib.sha256(pubkey + message +
signature[:32]).digest()
        return hmac.compare_digest(signature[32:], exp_p2)


#
================================================================
# 4. PURE BINARY ZERO-COPY TLV SERIALIZER (ZERO JSON / ZERO PARSING)
#
================================================================

class UniNodePayload:
    """
    Strictly typed binary serializer and Zero-Copy memoryview reader.
    Binary struct schemas (Little-Endian prefix \x3c):
      - 0x01 NOTARY_ANCHOR     : \x3c32s32s64sQIB (141 bytes exact)
      - 0x02 SEMANTIC_TENSOR   : \x3cNf (IEEE 754 float32 array, 4
bytes per element)
      - 0x03 KNOWLEDGE_GRAPH   : \x3cQQQeeI (32 bytes per triplet,
AVX2/NEON cache-aligned)
      - 0x04 TELEMETRY_MATRIX  : \x3cIf (8 bytes per sensor reading:
ID32 + float32)
      - 0x05 BEHAVIOR_CONTRACT : \x3cIBBHII32s (48 bytes exact
lifecycle & HVAC contract)
      - 0x06 GENESIS_TEXT      : Raw UTF-8 buffer (isolated
human-readable fallback)
      - 0x07 HOLO_WAVEFRONT    : \x3cffffHH (20 bytes per Pockels/MEMS
holographic voxel)
    """
    NOTARY_FMT    = "\x3c32s32s64sQIB"  # 141 bytes
    TRIPLET_FMT   = "\x3cQQQeeI"        # 32 bytes
    TELEMETRY_FMT = "\x3cIf"            # 8 bytes
    CONTRACT_FMT  = "\x3cIBBHII32s"     # 48 bytes
    HOLO_FMT      = "\x3cffffHH"        # 20 bytes

    @classmethod
    def pack_node(cls, node_type: UniNodeType, data: Any) -> bytes:
        if isinstance(data, (bytes, bytearray, memoryview)) and
```

```
node_type != UniNodeType.GENESIS_TEXT:
            return bytes(data)

        if node_type == UniNodeType.NOTARY_ANCHOR:
            status_byte = 1 if int(data.get("cooling_off_delta_t_sec",
0)) > 0 else 0
            return struct.pack(
                cls.NOTARY_FMT,
                bytes.fromhex(data["data_merkle_root_256"]),
                bytes.fromhex(data["signer_pubkey_ed25519"]),
                bytes.fromhex(data["signature_ed25519"]),
                int(data["timestamp_us"]),
                int(data["cooling_off_delta_t_sec"]),
                status_byte,
            )

        elif node_type == UniNodeType.SEMANTIC_TENSOR:
            count = len(data)
            return struct.pack(f"\x3c{count}f", *data)

        elif node_type == UniNodeType.KNOWLEDGE_GRAPH:
            buf = bytearray(len(data) * 32)
            offset = 0
            for item in data:
                s_id = semantic_id64(item["s"])
                p_id = semantic_id64(item["p"])
                o_id = semantic_id64(item["o"])
                weight = float(item.get("weight", 1.0))
                ego = float(item.get("ego_alignment", 0.0))
                flags = 1 if item.get("is_delta_conflict", False) else
0
                struct.pack_into(cls.TRIPLET_FMT, buf, offset, s_id,
p_id, o_id, weight, ego, flags)
                offset += 32
            return bytes(buf)

        elif node_type == UniNodeType.TELEMETRY_MATRIX:
            buf = bytearray(len(data) * 8)
            offset = 0
            for k, v in data.items():
                m_id = metric_id32(k)
                struct.pack_into(cls.TELEMETRY_FMT, buf, offset, m_id,
float(v))
                offset += 8
            return bytes(buf)

        elif node_type == UniNodeType.BEHAVIOR_CONTRACT:
            ui_val = data.get("ui_intent",
```

```
UIIntentCode.INTENT_WEIGHT_FOCUS)
            if isinstance(ui_val, str):
                ui_code = int(UIIntentCode.INTENT_WEIGHT_FOCUS) if
"Focus" in ui_val else 0
            else:
                ui_code = int(ui_val)
            media_hash = bytes.fromhex(data.get("media_sha256_hex",
"00" * 32))
            return struct.pack(
                cls.CONTRACT_FMT,
                int(data.get("ttl_sec", 3600)),
                int(data.get("zone",
MemoryZone.ZONE_1_ACTIVE_STREAM)),
                int(data.get("hvac_mode",
HVACMode.SOLAR_SHIELD_SUMMER)),
                ui_code,
                int(data.get("laser_pulse_cap_mw", 850)),
                int(data.get("reserved_flags", 0)),
                media_hash,
            )

        elif node_type == UniNodeType.GENESIS_TEXT:
            return data if isinstance(data, bytes) else
str(data).encode("utf-8")

        elif node_type == UniNodeType.HOLO_WAVEFRONT:
            buf = bytearray(len(data) * 20)
            offset = 0
            for v in data:
                struct.pack_into(
                    cls.HOLO_FMT,
                    buf,
                    offset,
                    float(v["x"]),
                    float(v["y"]),
                    float(v["z"]),
                    float(v["phase_2pi"]),
                    int(v["rgb565"]),
                    int(v["pulse_mw"]),
                )
                offset += 20
            return bytes(buf)

        raise ValueError(f"Unsupported UniNodeType: {node_type}")

    @classmethod
    def unpack_node(cls, node_type: UniNodeType, raw_buf: Union[bytes,
memoryview]) -> Any:
```

```
        view = memoryview(raw_buf)

        if node_type == UniNodeType.NOTARY_ANCHOR:
            root, pub, sig, ts_us, delta_t, st =
struct.unpack(cls.NOTARY_FMT, view)
            return {
                "data_merkle_root_256": root.hex(),
                "signer_pubkey_ed25519": pub.hex(),
                "signature_ed25519": sig.hex(),
                "timestamp_us": ts_us,
                "cooling_off_delta_t_sec": delta_t,
                "status": "PENDING_DELTA_T_CONFIRMATION" if st == 1
else "FINALIZED",
            }

        elif node_type == UniNodeType.SEMANTIC_TENSOR:
            # Zero-Copy memory projection directly to IEEE 754 float32
view
            return list(view.cast("f"))

        elif node_type == UniNodeType.KNOWLEDGE_GRAPH:
            return list(cls.iter_triplets_zero_copy(view))

        elif node_type == UniNodeType.TELEMETRY_MATRIX:
            out: Dict[str, float] = {}
            for m_id, val in struct.iter_unpack(cls.TELEMETRY_FMT,
view):
                out[resolve_metric_id32(m_id)] = round(val, 4)
            return out

        elif node_type == UniNodeType.BEHAVIOR_CONTRACT:
            ttl, zone, hvac, ui_code, mw, flags, m_hash =
struct.unpack(cls.CONTRACT_FMT, view)
            return {
                "ttl_sec": ttl,
                "zone": zone,
                "hvac_mode": HVACMode(hvac).name,
                "ui_intent_code": f"0x{ui_code:04x}",
                "laser_pulse_cap_mw": mw,
                "reserved_flags": flags,
                "media_sha256_hex": m_hash.hex(),
            }

        elif node_type == UniNodeType.GENESIS_TEXT:
            return bytes(view).decode("utf-8")

        elif node_type == UniNodeType.HOLO_WAVEFRONT:
            voxels = []
```

```
            for x, y, z, ph, rgb, mw in
struct.iter_unpack(cls.HOLO_FMT, view):
                voxels.append({
                    "x": round(x, 4),
                    "y": round(y, 4),
                    "z": round(z, 4),
                    "phase_2pi": round(ph, 6),
                    "rgb565": rgb,
                    "pulse_mw": mw,
                })
            return voxels

        raise ValueError(f"Unknown UniNodeType during unpacking:
{node_type}")

    @classmethod
    def iter_triplets_zero_copy(cls, raw_view: memoryview) ->
Iterator[Dict[str, Any]]:
        """Iterates over 32-byte binary triplets directly in
memoryview without string parsing."""
        for s_id, p_id, o_id, weight, ego, flags in
struct.iter_unpack(cls.TRIPLET_FMT, raw_view):
            yield {
                "s": resolve_semantic_id64(s_id),
                "p": resolve_semantic_id64(p_id),
                "o": resolve_semantic_id64(o_id),
                "s_id64": s_id,
                "p_id64": p_id,
                "o_id64": o_id,
                "weight": round(weight, 4),
                "ego_alignment": round(ego, 4),
                "is_delta_conflict": bool(flags & 1),
            }


#
================================================================================
# 5. UNIVERSAL .UNI CONTAINER ENGINE (O(1) GATE, MERKLE, ZERO-COPY)
#
================================================================================

class UniUniversalContainer:
    """
    32-Byte Little-Endian UniHeader Layout ('\x3c4sHHIIQ4s4s'):
      00..03 (4B) MAGIC_BYTES   : b'UNI8'
      04..05 (2B) PROTO_FLAGS   : Version (0x0820) & mode bitmask
      06..07 (2B) NODE_BITMAP   : Active node bitmask (nodes 0..15)
      08..11 (4B) PAYLOAD_LIMIT : Exact encrypted node stream size in
```

```
bytes
      12..15 (4B) SESSION_SALT  : 32-bit TRNG cryptographic salt
      16..23 (8B) TIMESTAMP_US  : 64-bit UTC microsecond counter
      24..27 (4B) MERKLE_TAG    : Truncated 4-byte Data Merkle root
anchor
      28..31 (4B) HEADER_ECC    : 4-byte Blake2s-32 header integrity
syndrome
    """
    HEADER_FMT    = "\x3c4sHHIIQ4s4s"
    HEADER_28_FMT = "\x3c4sHHIIQ4s"
    NODE_DESC_FMT = "\x3cHBI"
    HEADER_SIZE   = 32
    MAGIC         = b"UNI8"
    NODE_OVERHEAD = 35  # 7B descriptor + 12B IV + 16B GCM Tag

    @staticmethod
    def leaf_hash(node_id: int, node_type: int, raw_bytes:
Union[bytes, memoryview]) -> bytes:
        h = hashlib.sha256(b"UNI8_LEAF:" + struct.pack("\x3cHB",
node_id, node_type))
        h.update(raw_bytes)
        return h.digest()

    @staticmethod
    def branch_hash(left: bytes, right: bytes) -> bytes:
        return hashlib.sha256(b"UNI8_BRANCH:" + left + right).digest()

    @classmethod
    def build_merkle_tree(
        cls, node_ids: List[int], leaves: List[bytes]
    ) -> Tuple[bytes, Dict[int, List[Dict[str, str]]]]:
        """Builds a 256-bit SHA-256 Merkle Tree, returning the root
and per-node Branch Proofs."""
        if not leaves:
            return b"\x00" * 32, {}
        proofs: Dict[int, List[Dict[str, str]]] = {nid: [] for nid in
node_ids}
        positions = {nid: idx for idx, nid in enumerate(node_ids)}
        level = list(leaves)

        while len(level) > 1:
            if len(level) % 2 == 1:
                level.append(level[-1])
            for nid, pos in positions.items():
                sib = level[pos + 1] if pos % 2 == 0 else level[pos -
1]
                side = "right" if pos % 2 == 0 else "left"
                proofs[nid].append({"side": side, "hash": sib.hex()})
```

```
                positions[nid] = pos // 2
            level = [cls.branch_hash(level[i], level[i + 1]) for i in
range(0, len(level), 2)]

        return level[0], proofs

    @classmethod
    def verify_header_o1(cls, buffer: Union[bytes, memoryview]) ->
bool:
        """Constant-time O(1) validation of the 32-byte header and
payload size limit."""
        view = memoryview(buffer)
        if cls.HEADER_SIZE > len(view):
            return False
        magic, flags, bitmap, payload_limit, salt, ts_us, m_tag, ecc =
struct.unpack(
            cls.HEADER_FMT, view[:cls.HEADER_SIZE]
        )
        if magic != cls.MAGIC:
            return False
        actual_rem = len(view) - cls.HEADER_SIZE
        if (flags & ProtoFlags.RS_FEC_APPENDED) and actual_rem >=
payload_limit:
            pass
        elif actual_rem != payload_limit:
            return False
        expected_ecc = hashlib.blake2s(view[:28],
digest_size=4).digest()
        return hmac.compare_digest(ecc, expected_ecc)

    @classmethod
    def create_container(
        cls,
        nodes_input: Dict[int, Tuple[UniNodeType, Any]],
        keyring: SovereignKeyring,
        cooling_off_seconds: int = 0,
        under_duress: bool = False,
        is_delta_frame: bool = False,
        spatial_broadcast: bool = False,
        rs_fec_tail: bytes = b"",
    ) -> bytes:
        """Assembles an encrypted pure-binary .uni container with
39-byte Header+NodeDesc AAD."""
        if 0 in nodes_input:
            raise ValueError("Node ID 0 is reserved for the system
NOTARY_ANCHOR node.")
        for nid in nodes_input:
            if not (15 >= nid >= 1):
```

```
                raise ValueError(f"Node ID {nid} exceeds 16-bit
NODE_BITMAP bounds (1..15).")

        session_salt = struct.unpack("\x3cI", os.urandom(4))[0]
        timestamp_us = int(time.time() * 1_000_000)

        sorted_user_ids = sorted(nodes_input.keys())
        packed_nodes: Dict[int, Tuple[UniNodeType, bytes]] = {}
        user_leaves: List[bytes] = []

        for nid in sorted_user_ids:
            ntype, content = nodes_input[nid]
            raw_b = UniNodePayload.pack_node(ntype, content)
            packed_nodes[nid] = (ntype, raw_b)
            user_leaves.append(cls.leaf_hash(nid, int(ntype), raw_b))

        data_merkle_root, _ = cls.build_merkle_tree(sorted_user_ids,
user_leaves)

        flags = ProtoFlags.VERSION_8_2 | ProtoFlags.TAMPER_MESH_INTACT
        if spatial_broadcast:
            flags |= ProtoFlags.SPATIAL_FRUSTUM_BROADCAST
        if rs_fec_tail:
            flags |= ProtoFlags.RS_FEC_APPENDED

        if is_delta_frame:
            flags |= ProtoFlags.STREAM_DELTA_FRAME
            all_ids = sorted_user_ids
        else:
            notary_msg = data_merkle_root + struct.pack("\x3cQI",
timestamp_us, cooling_off_seconds)
            signer_pubkey, signature = keyring.sign_data(notary_msg,
under_duress=under_duress)
            notary_dict = {
                "data_merkle_root_256": data_merkle_root.hex(),
                "signer_pubkey_ed25519": signer_pubkey.hex(),
                "signature_ed25519": signature.hex(),
                "timestamp_us": timestamp_us,
                "cooling_off_delta_t_sec": cooling_off_seconds,
            }
            notary_raw_b =
UniNodePayload.pack_node(UniNodeType.NOTARY_ANCHOR, notary_dict)
            packed_nodes[0] = (UniNodeType.NOTARY_ANCHOR,
notary_raw_b)
            all_ids = [0] + sorted_user_ids

        bitmap = sum(pow(2, nid) for nid in all_ids)
        payload_limit = sum(cls.NODE_OVERHEAD +
```

```
len(packed_nodes[nid][1]) for nid in all_ids)

        header_28b = struct.pack(
            cls.HEADER_28_FMT,
            cls.MAGIC,
            int(flags),
            bitmap,
            payload_limit,
            session_salt,
            timestamp_us,
            data_merkle_root[:4],
        )
        ecc = hashlib.blake2s(header_28b, digest_size=4).digest()
        full_header_32b = header_28b + ecc

        salt_bytes = struct.pack("\x3cI", session_salt)
        encrypted_chunks: List[bytes] = []

        for nid in all_ids:
            ntype, raw_b = packed_nodes[nid]
            node_desc = struct.pack(cls.NODE_DESC_FMT, nid,
int(ntype), len(raw_b))
            iv = hashlib.sha256(salt_bytes + struct.pack("\x3cH", nid)
+ os.urandom(8)).digest()[:12]
            node_key = hkdf_sha256(keyring.master_encryption_secret,
salt_bytes, f"NODE_{nid}".encode())
            ct, tag = aead_encrypt(node_key, iv, raw_b,
aad=full_header_32b + node_desc)
            encrypted_chunks.append(node_desc + iv + tag + ct)

        return full_header_32b + b"".join(encrypted_chunks) +
rs_fec_tail

    @classmethod
    def inspect_and_route(
        cls, uni_binary: Union[bytes, memoryview], keyring:
SovereignKeyring
    ) -> Dict[str, Any]:
        """Zero-Copy container inspection via memoryview slicing,
Notary verification & Zone routing."""
        view = memoryview(uni_binary)
        if not cls.verify_header_o1(view):
            raise ValueError("O(1) Hardware Gate: Invalid header or
payload length mismatch!")

        full_header_32b = bytes(view[:cls.HEADER_SIZE])
        magic, flags, bitmap, payload_limit, session_salt, ts_us,
m_tag, ecc = struct.unpack(
```

```
            cls.HEADER_FMT, full_header_32b
        )
        salt_bytes = struct.pack("\x3cI", session_salt)

        stream_end = cls.HEADER_SIZE + payload_limit
        offset = cls.HEADER_SIZE
        unpacked_nodes: Dict[int, Dict[str, Any]] = {}

        while stream_end > offset:
            node_desc = bytes(view[offset:offset + 7])
            nid, ntype_int, raw_len = struct.unpack(cls.NODE_DESC_FMT,
node_desc)
            iv_view = view[offset + 7:offset + 19]
            tag_view = view[offset + 19:offset + 35]
            ct_view = view[offset + 35:offset + 35 + raw_len]
            offset += 35 + raw_len

            node_key = hkdf_sha256(keyring.master_encryption_secret,
salt_bytes, f"NODE_{nid}".encode())
            raw_b = aead_decrypt(node_key, iv_view, ct_view, tag_view,
aad=full_header_32b + node_desc)
            ntype = UniNodeType(ntype_int)
            unpacked_nodes[nid] = {
                "type": ntype.name,
                "raw_size_bytes": len(raw_b),
                "data": UniNodePayload.unpack_node(ntype,
memoryview(raw_b)),
            }

        if flags & ProtoFlags.STREAM_DELTA_FRAME:
            return {
                "frame_mode": "FAST_DELTA_FRAME_120HZ",
                "public_signature_valid": True,
                "assigned_memory_zone":
MemoryZone.ZONE_1_ACTIVE_STREAM.name,
                "internal_security_verdict": "VERIFIED_SESSION_AEAD:
Header-bound GCM Delta Frame.",
                "nodes": unpacked_nodes,
            }

        notary = unpacked_nodes[0]["data"]
        signer_pub = bytes.fromhex(notary["signer_pubkey_ed25519"])
        signature = bytes.fromhex(notary["signature_ed25519"])
        data_root = bytes.fromhex(notary["data_merkle_root_256"])
        delta_t = notary["cooling_off_delta_t_sec"]

        signed_msg = data_root + struct.pack("\x3cQI",
notary["timestamp_us"], delta_t)
```

```
        is_sig_valid = SovereignKeyring.verify_signature(signer_pub,
signed_msg, signature)

        if hmac.compare_digest(signer_pub, keyring.duress_pubkey):
            zone = MemoryZone.ZONE_3_QUARANTINE
            verdict = "SILENT_DURESS_ALERT: Coercion key detected!
Isolated in Zone 3 Quarantine."
        elif not is_sig_valid:
            zone = MemoryZone.ZONE_3_QUARANTINE
            verdict = "INVALID_SIGNATURE: Cryptographic verification
failed."
        else:
            zone = MemoryZone.ZONE_1_ACTIVE_STREAM
            verdict = "VERIFIED_SOVEREIGN: Authentic Primary
Signature."

        return {
            "frame_mode": "KEYFRAME_NOTARIZED",
            "public_signature_valid": is_sig_valid,
            "assigned_memory_zone": zone.name,
            "internal_security_verdict": verdict,
            "nodes": unpacked_nodes,
        }

    @classmethod
    def export_selective_slice(
        cls, uni_binary: Union[bytes, memoryview], keyring:
SovereignKeyring, target_node_id: int
    ) -> Dict[str, Any]:
        """Exports a single decrypted user node accompanied by its
Merkle Branch Proof & Node 0 Notary Seal."""
        if target_node_id == 0:
            raise ValueError("Node 0 is the Notary Anchor; specify a
user node (1..15) to disclose.")
        view = memoryview(uni_binary)
        if not cls.verify_header_o1(view):
            raise ValueError("Container failed O(1) header check.")

        full_header_32b = bytes(view[:cls.HEADER_SIZE])
        _, flags, bitmap, payload_limit, session_salt, _, _, _ =
struct.unpack(
            cls.HEADER_FMT, full_header_32b
        )
        if not (bitmap & pow(2, target_node_id)):
            raise ValueError(f"Node {target_node_id} is not present in
container.")

        salt_bytes = struct.pack("\x3cI", session_salt)
```

```
        offset = cls.HEADER_SIZE
        stream_end = cls.HEADER_SIZE + payload_limit
        user_ids: List[int] = []
        user_leaves: List[bytes] = []
        target_raw = b""
        target_type = UniNodeType.GENESIS_TEXT
        notary_data: Dict[str, Any] = {}

        while stream_end > offset:
            node_desc = bytes(view[offset:offset + 7])
            nid, ntype_int, raw_len = struct.unpack(cls.NODE_DESC_FMT,
node_desc)
            iv_view = view[offset + 7:offset + 19]
            tag_view = view[offset + 19:offset + 35]
            ct_view = view[offset + 35:offset + 35 + raw_len]
            offset += 35 + raw_len

            node_key = hkdf_sha256(keyring.master_encryption_secret,
salt_bytes, f"NODE_{nid}".encode())
            raw_b = aead_decrypt(node_key, iv_view, ct_view, tag_view,
aad=full_header_32b + node_desc)
            if nid == 0:
                notary_data =
UniNodePayload.unpack_node(UniNodeType.NOTARY_ANCHOR,
memoryview(raw_b))
            else:
                user_ids.append(nid)
                user_leaves.append(cls.leaf_hash(nid, ntype_int,
raw_b))
                if nid == target_node_id:
                    target_raw = raw_b
                    target_type = UniNodeType(ntype_int)

        data_root_256, proofs = cls.build_merkle_tree(user_ids,
user_leaves)
        return {
            "node_id": target_node_id,
            "node_type": int(target_type),
            "node_type_name": target_type.name,
            "decoded_data": UniNodePayload.unpack_node(target_type,
memoryview(target_raw)),
            "raw_payload_hex": target_raw.hex(),
            "data_merkle_root_256": data_root_256.hex(),
            "merkle_branch_proof": proofs[target_node_id],
            "notary_anchor": notary_data,
        }

    @classmethod
```

```
    def verify_selective_slice(
        cls, exported_slice: Dict[str, Any], expected_pubkey_hex:
Optional[str] = None
    ) -> bool:
        """Verifies a single exported node against the 256-bit Merkle
root AND the author's Ed25519 signature."""
        curr = cls.leaf_hash(
            exported_slice["node_id"],
            exported_slice["node_type"],
            bytes.fromhex(exported_slice["raw_payload_hex"]),
        )
        for step in exported_slice["merkle_branch_proof"]:
            sib = bytes.fromhex(step["hash"])
            curr = cls.branch_hash(curr, sib) if step["side"] ==
"right" else cls.branch_hash(sib, curr)

        notary = exported_slice["notary_anchor"]
        if not hmac.compare_digest(curr.hex(),
notary["data_merkle_root_256"]):
            return False

        if expected_pubkey_hex and not
hmac.compare_digest(notary["signer_pubkey_ed25519"],
expected_pubkey_hex):
            return False

        signed_msg = (
            bytes.fromhex(notary["data_merkle_root_256"])
            + struct.pack("\x3cQI", notary["timestamp_us"],
notary["cooling_off_delta_t_sec"])
        )
        return SovereignKeyring.verify_signature(
            bytes.fromhex(notary["signer_pubkey_ed25519"]),
            signed_msg,
            bytes.fromhex(notary["signature_ed25519"]),
        )


#
================================================================
# 6. ZONE 2 SEMANTIC TRIAGE (ZERO-COPY BINARY GRAPH MERGING IN RAM)
#
================================================================

class Zone2SemanticTriage:
    """Background vector clustering and in-RAM binary Knowledge Graph
merging with Delta superposition."""
```

```
    @staticmethod
    def cosine_distance(
        vec_a: Union[List[float], memoryview], vec_b:
Union[List[float], memoryview]
    ) -> float:
        a_floats = vec_a.cast("f") if isinstance(vec_a, memoryview)
else vec_a
        b_floats = vec_b.cast("f") if isinstance(vec_b, memoryview)
else vec_b
        dot = sum(x * y for x, y in zip(a_floats, b_floats))
        norm_a = math.sqrt(sum(x * x for x in a_floats))
        norm_b = math.sqrt(sum(y * y for y in b_floats))
        if norm_a == 0.0 or norm_b == 0.0:
            return 1.0
        return round(1.0 - (dot / (norm_a * norm_b)), 6)

    @classmethod
    def merge_binary_graphs_in_ram(cls, view_a: memoryview, view_b:
memoryview) -> bytes:
        """
        Merges two 32-byte aligned binary Knowledge Graph buffers
directly in RAM
        by comparing 64-bit integer IDs (0% string comparison
overhead).
        """
        fmt = UniNodePayload.TRIPLET_FMT
        grouped: Dict[Tuple[int, int], List[Tuple[int, float, float]]]
= {}
        for view in (view_a, view_b):
            for s_id, p_id, o_id, w, ego, _ in struct.iter_unpack(fmt,
view):
                key = (s_id, p_id)
                grouped.setdefault(key, [])
                if not any(existing[0] == o_id for existing in
grouped[key]):
                    grouped[key].append((o_id, w, ego))

        out_buf = bytearray()
        for (s_id, p_id), edges in grouped.items():
            conflict_flag = 1 if len(edges) > 1 else 0
            for o_id, w, ego in edges:
                out_buf.extend(struct.pack(fmt, s_id, p_id, o_id, w,
ego, conflict_flag))
        return bytes(out_buf)

    @classmethod
    def merge_knowledge_graphs(
        cls,
```

```
        graph_a: Union[List[Dict[str, Any]], bytes, memoryview],
        graph_b: Union[List[Dict[str, Any]], bytes, memoryview],
    ) -> List[Dict[str, Any]]:
        """Packs (if needed) and merges two graphs via the Zero-Copy
binary RAM engine."""
        raw_a = (
            UniNodePayload.pack_node(UniNodeType.KNOWLEDGE_GRAPH,
graph_a)
            if isinstance(graph_a, list)
            else bytes(graph_a)
        )
        raw_b = (
            UniNodePayload.pack_node(UniNodeType.KNOWLEDGE_GRAPH,
graph_b)
            if isinstance(graph_b, list)
            else bytes(graph_b)
        )
        merged_bytes =
cls.merge_binary_graphs_in_ram(memoryview(raw_a), memoryview(raw_b))
        return
list(UniNodePayload.iter_triplets_zero_copy(memoryview(merged_bytes)))


#
================================================================
# SELF-TEST & VERIFICATION SUITE (ALL 5 CORE, ZERO-COPY & HOLO TESTS)
#
================================================================
if __name__ == "__main__":

print("================================================================
==========")
    print(" .uni Universal Reference SDK v8.2 - Pure Binary Zero-Copy
Suite")
    print(f" Crypto Backend : {'Hardware AES-256-GCM & Ed25519' if
HAS_CRYPTO_LIB else 'Stdlib AEAD Fallback'}")
    print(" JSON Dependency: ELIMINATED (0% Text Parsing inside
Operational Nodes)")

print("================================================================
==========\n")

    keyring =
SovereignKeyring(master_seed=b"SOVEREIGN_ROOT_SEED_2026_V8_2")

    sample_nodes = {
        1: (UniNodeType.SEMANTIC_TENSOR, [0.82, 0.14, -0.33, 0.91]),
        2: (UniNodeType.KNOWLEDGE_GRAPH, [
```

```
            {"s": "MZM-Pod", "p": "uses_memory", "o": "Volatile_SRAM",
"weight": 0.99, "ego_alignment": 0.95},
            {"s": "Dump_Valve", "p": "triggers_at", "o": "1.2_atm",
"weight": 0.95, "ego_alignment": 0.90},
        ]),
        3: (UniNodeType.TELEMETRY_MATRIX, {"carotid_IMT_mm": 0.72,
"VAT_SAT_ratio": 0.41, "PWV_ms": 6.8}),
        4: (UniNodeType.BEHAVIOR_CONTRACT, {
            "ttl_sec": 3600,
            "zone": MemoryZone.ZONE_1_ACTIVE_STREAM,
            "hvac_mode": HVACMode.SOLAR_SHIELD_SUMMER,
            "ui_intent": UIIntentCode.INTENT_WEIGHT_FOCUS,
            "laser_pulse_cap_mw": 850,
        }),
        5: (UniNodeType.GENESIS_TEXT, "# Session Summary\nVascular and
metabolic parameters within normal range."),
        7: (UniNodeType.HOLO_WAVEFRONT, [
            {"x": 12.5, "y": 40.0, "z": 2.5, "phase_2pi": 3.14159,
"rgb565": 0xF800, "pulse_mw": 850},
            {"x": 12.6, "y": 40.0, "z": 2.8, "phase_2pi": 1.57079,
"rgb565": 0x07E0, "pulse_mw": 820},
        ]),
    }

    # Test 1: Pure Binary Container with 12-hour (43,200s) Delta-T
Cooling-Off Timelock
    uni_normal = UniUniversalContainer.create_container(
        sample_nodes, keyring, cooling_off_seconds=43200,
under_duress=False
    )
    inspected_normal =
UniUniversalContainer.inspect_and_route(memoryview(uni_normal),
keyring)
    print(f"[Test 1] Pure Binary Container Packed ({len(uni_normal)}
bytes total)")
    print(f"         Node 0 Binary Size     :
{inspected_normal['nodes'][0]['raw_size_bytes']}B (Fixed 141B)")
    print(f"         Node 2 Binary KG Size  :
{inspected_normal['nodes'][2]['raw_size_bytes']}B (2 triplets x 32B)")
    print(f"         Node 4 Contract Size   :
{inspected_normal['nodes'][4]['raw_size_bytes']}B (Fixed 48B)")
    print(f"         Public Signature Valid :
{inspected_normal['public_signature_valid']}")
    print(f"         Assigned Memory Zone   :
{inspected_normal['assigned_memory_zone']}")
    print(f"         Notary Timelock Status :
{inspected_normal['nodes'][0]['data']['status']}\n")
```

```
    # Test 2: Coerced Signature (Steganographic Duress Key)
    uni_duress = UniUniversalContainer.create_container(
        sample_nodes, keyring, cooling_off_seconds=0,
under_duress=True
    )
    inspected_duress =
UniUniversalContainer.inspect_and_route(memoryview(uni_duress),
keyring)
    print("[Test 2] Coerced Container (Signed with Duress Key)")
    print(f"         External Verifier Sees : Signature Valid =
{inspected_duress['public_signature_valid']}")
    print(f"         Sovereign Node Action  :
{inspected_duress['internal_security_verdict']}\n")

    # Test 3: Selective Single-Node Disclosure (Node 3) with Merkle
Branch & Ed25519 Verification
    telemetry_slice = UniUniversalContainer.export_selective_slice(
        memoryview(uni_normal), keyring, target_node_id=3
    )
    slice_verified = UniUniversalContainer.verify_selective_slice(
        telemetry_slice,
expected_pubkey_hex=keyring.primary_pubkey.hex()
    )
    print("[Test 3] Selective Single-Node Export (Node 3:
TELEMETRY_MATRIX)")
    print(f"         Disclosed Metrics      :
{telemetry_slice['decoded_data']}")
    print(f"         Merkle + Ed25519 Proof : {slice_verified} (Nodes
1, 2, 4, 5, 7 remain unexposed)\n")

    # Test 4: Zone 2 Semantic Triage & In-RAM Delta Conflict
Superposition
    c_dist = Zone2SemanticTriage.cosine_distance([0.82, 0.14, -0.33,
0.91], [0.80, 0.15, -0.31, 0.89])
    merged = Zone2SemanticTriage.merge_knowledge_graphs(
        sample_nodes[2][1],
        [{"s": "Dump_Valve", "p": "triggers_at", "o": "1.5_atm",
"weight": 0.80, "ego_alignment": 0.85}],
    )
    print(f"[Test 4] Zone 2 In-RAM Semantic Triage | Cosine Distance =
{c_dist} (threshold 0.15)")
    for triplet in merged:
        delta_tag = " [DELTA CONFLICT SUPERPOSITION]" if
triplet["is_delta_conflict"] else ""
        print(f"         [{triplet['s']}] =({triplet['p']})==>
[{triplet['o']}] (w={triplet['weight']}){delta_tag}")
    print()
```

```
    # Test 5: Holo-Frame 120Hz Delta-Stream & Reed-Solomon Facade
Broadcast
    delta_pkt = UniUniversalContainer.create_container(
        {7: sample_nodes[7]},
        keyring,
        is_delta_frame=True,
        spatial_broadcast=True,
        rs_fec_tail=b"\xAA" * 32,
    )
    res_df =
UniUniversalContainer.inspect_and_route(memoryview(delta_pkt),
keyring)
    print(f"[Test 5] Holo-Frame 120Hz Delta Packet ({len(delta_pkt)}
bytes inc. 32B RS-FEC tail)")
    print(f"         Frame Mode             : {res_df['frame_mode']}")
    print(f"         Decoded Wavefront      :
{len(res_df['nodes'][7]['data'])} Pockels/MEMS voxels (20B/voxel)")
```